TEAM TERRAMODEL

Phase 03 Deliverable

Trained Model, Evaluation, and Limitation Fixes

Weeks 7 through 10: What We Built, What It Does, What Is Next

Prepared by

Robert B. Thompson and Cesar Noriega

October 29, 2026

Repository: github.com/w212444150/TerraModel_WarehouseCyberSec

Latest commit at submission: e4c9bea

Executive Summary

TerraModel is my capstone project with Cesar Noriega. We are building a multi-agent AI system that detects cyber threats against the robots, PLCs, and industrial sensors in an automated warehouse. Phase 03 covers Weeks 7 through 10 of the capstone, the Trained Model with Evaluation Report deliverable.

In this phase we replaced the unsupervised Phase 02 baselines with supervised machine learning models, tuned them with Optuna, added SHAP explainability, mapped every detected threat to four defensive cybersecurity frameworks, and then went further to fix the four honest limitations the first pass left on the table.

The result is a reproducible working model. One command (make phase03-fixes) runs the whole pipeline end to end in about 90 seconds on a laptop. Every number in this deliverable comes from running the committed code. I ran it on Google Colab on October 29, 2026, and the live CISA Known Exploited Vulnerabilities feed returned 1,764 real actively-exploited CVEs that the models now train against.

1. What We Built

1.1 The Model System

Three detection agents and one orchestrator work together.

Component	Job	Algorithm	Phase 03 Status
Agent 1	Detect anomalies in OT network traffic	XGBoost (Optuna-tuned)	F1 0.43 net-only; F1 1.00 with PLC
Agent 2	Rank unpatched ICS vulns to fix first	Logistic Regression on CISA KEV labels	CV ROC-AUC 0.61; precision@10 = 1.00
Agent 3	Detect spoofed robot commands and deviations	Random Forest (300 trees, balanced)	F1 0.79; Recall 0.78; PR-AUC 0.90
Orchestrator	Correlate alerts; tag with ATT&CK and framework controls	Rule-based mapping + enrichment	7 ATT&CK techniques; 4 frameworks

1.2 Supporting Infrastructure

- Pure-Python synthetic warehouse OT data generator producing 1.9 million labeled rows per run
- Enhanced realistic generator that injects documented real-OT quirks (Modbus retry bursts, robot joint actuator wear, maintenance windows, legacy protocol noise)
- Live CISA ATT&CK for ICS v18 downloader (STIX bundle) with 7 target techniques resolved
- Live CISA ICS Advisories fetcher with 25-entry fallback for restricted networks
- Live CISA Known Exploited Vulnerabilities (KEV) catalog fetcher pulling 1,764 real entries from the live feed on Colab
- Public HAI dataset loader (collectors/load_hai.py) that maps 50+ labeled real OT attack campaigns onto the TerraModel schema for Phase 04 real-data runs
- Cybersecurity framework mapping module tying every ATT&CK technique to NIST CSF 2.0, IEC 62443-3-3, NIST SP 800-82 Rev. 3, and MITRE D3FEND
- Lightweight JSON experiment tracker (one file per run under reports/phase03/experiments)

- GitHub Actions CI running the full Phase 02 + Phase 03 pipeline on every push
- Full pytest suite with 16 passing tests

2. What the Model Can Do Today

2.1 Trained Capabilities

The system takes OT telemetry (synthetic now, real OpenPLC or HAI in Phase 04) and does five things end to end.

- **Detects** anomalies in OT network traffic in near real time (Agent 1). Trained on labeled attack windows, it catches denial-of-service floods, unauthorized Modbus commands, and remote-service exploit attempts.
- **Ranks** which unpatched ICS vulnerabilities to fix first (Agent 2). Trained on real CISA Known Exploited Vulnerabilities labels, it reads the live advisory feed and tells your team which CVEs are actually being exploited in the wild.
- **Monitors** robot command integrity (Agent 3). Trained on labeled trajectory and command-sequence features, it catches spoofed commands, unsafe path deviations, and unauthorized command sources with 78% recall at 80% precision.
- **Correlates** the three agent streams through the orchestrator, deduplicating alerts and tagging each with the specific MITRE ATT&CK for ICS technique that fired (T0836, T0855, T0832, T0814, T0859, T0866, T0863).
- **Explains** each alert in defensive-framework language the audit team already knows: NIST CSF 2.0 subcategories, IEC 62443-3-3 System Requirements, NIST SP 800-82 Rev. 3 control sections, and MITRE D3FEND countermeasure IDs.

2.2 Real Metrics on the Test Set

Every number below came from running the committed code with random seed 42. The metrics file lives at reports/phase03/.

Agent	Metric	Phase 02 Baseline	Phase 03	Phase 03 + Fix
Agent 1	F1	0.242 (IF)	0.434 (XGB net-only)	1.000 (net + PLC)
Agent 1	PR-AUC	0.545	0.660	1.000
Agent 1	Recall	0.156	0.333	1.000
Agent 2	CV ROC-AUC	n/a	1.000 (tautological)	0.611 (real KEV)
Agent 2	Top-10 OT	7 of 10	10 of 10 (derived)	KEV precision@10 = 1.00
Agent 3	F1	0.374 (Mah.)	0.793 (RF)	same
Agent 3	Recall	0.244	0.781	same
Agent 3	PR-AUC	0.712	0.897	same

Honest note on the 1.000 scores. Our synthetic generator pushes T0836 attacks to 1.6x setpoint, which is trivially separable once Agent 1 can see PLC data. The score reflects the fix working against our generator. On real HAI data we expect those perfect scores to settle into the 0.80 to 0.90 range, which is still a huge jump from the 0.49 ceiling.

2.3 What It Does Not Do Yet

- No live packet capture off a real network. Agents read Parquet files from the data pipeline; a Phase 04 adapter will subscribe to a real pcap source

- No FastAPI service or Streamlit operator dashboard. Everything runs as scripts today. Phase 04 wraps the orchestrator in a REST API and ships a web UI
- No deep sequence model (LSTM, Autoencoder). We stayed with tree-based classifiers because they extract the signal from our window-aggregated features without needing GPU. Phase 04 adds a PyTorch LSTM for per-packet sequence detection
- No real Factory I/O or ROS 2 integration. The synthetic generator stands in. Phase 04 adds the real OpenPLC bridge

3. Four Limitations and How We Fixed Them

The first Phase 03 pass left four honest limitations in the report. We applied fixes for all four and committed them. Each fix is backed by real test-set numbers.

3.1 Limitation 1: Agent 1 F1 ceiling near 0.49

The problem. Agent 1 only saw network flow summaries. Two attack techniques (T0836 Modify Parameter, T0832 Manipulation of View) hit the PLC and HMI directly without changing network behavior. Agent 1 was being graded on attacks it had no visibility into.

The fix. pipeline/features_augmented.py joins per-second PLC statistics (mean, std, range, setpoint delta, out-of-band fraction) into Agent 1 input. The last two features directly expose T0836 and T0832 attacks.

The result. On our synthetic data, network-only F1 was 0.525, PR-AUC 0.657. Network plus PLC: F1 1.000, PR-AUC 1.000. On real data expect 0.80 to 0.90.

3.2 Limitation 2: Agent 2 CV ROC-AUC was tautological

The problem. The target label (is_ot_critical) was derived from asset_type, which was also an input feature. The model trivially learned the lookup. 3-fold CV ROC-AUC of 1.0 did not prove generalization.

The fix. collectors/fetch_kev.py pulls the CISA Known Exploited Vulnerabilities catalog, a free, public, daily-updated feed of CVEs being actively exploited in the wild. agents/agent2_assessment/phase03_kev.py joins our advisories to KEV on (vendor, product) and trains on the real external label is_kev_match.

The result. 3-fold CV ROC-AUC dropped from the fake 1.000 to a real 0.611 plus or minus 0.134. CV PR-AUC 0.818. Precision at 10 on real labels: 1.000. Learned coefficients now make sense: CVSS score (+0.99), robot controller (+1.04), engineering workstation (-0.35).

3.3 Limitation 3: Synthetic data ceiling

The problem. Our Python generator produced data shaped like OT telemetry but did not model vendor-specific timing, protocol errors, retry storms, or maintenance windows.

The fix, Track A. collectors/generate_synthetic_realistic.py injects four documented real-OT quirks: Modbus retry bursts, robot joint actuator wear, maintenance windows, and legacy protocol noise. Sources: Dragos 2024 OT Year in Review, SANS 2024 ICS/OT Survey, Nozomi H2 2024, NIST SP 800-82 Rev. 3 Appendix D.

The fix, Track B. collectors/load_hai.py maps the public HAI 23.05 dataset onto our TerraModel schema. HAI is published by the Korean National Security Research Institute, free for research, with 50+ labeled real attack campaigns from a Hardware-In-the-Loop ICS testbed. Our build sandbox blocked Git LFS so the loader is ready but not run there. On Colab the three-line pull works, then the whole Phase 03 pipeline runs against real data.

The result. Realism injections do not break existing models (Agent 3 F1 stays at 0.79, which proves it is robust to the injected noise). Agent 1 network-only F1 moves 0.47 to 0.53 (small bump; realism exposes more signal in some attack types).

3.4 Limitation 4: 25-advisory CISA fallback

The problem. Our build sandbox blocked cisa.gov, so Agent 2 trained on a 25-advisory fallback sample.

The fix. On a normal network the live feed returns thousands. On my Colab run on October 29, 2026, the KEV live fetch returned 1,764 real entries. No code change needed.

4. How to Reproduce Everything

Any grader with Python 3.11 or newer and pip can reproduce every number in this deliverable in about 90 seconds on a laptop. Full instructions in the repo README.

```
git clone https://github.com/w212444150/TerraModel_WarehouseCyberSec.git
cd TerraModel_WarehouseCyberSec
python3 -m venv venv && source venv/bin/activate
make setup           # install dependencies
make data            # fetch threat data + generate + clean + features + split
make baseline        # Phase 02 unsupervised baselines
make phase03         # Phase 03 supervised models + framework + orchestrator
make phase03-fixes   # Limitation fixes: realism boost + augmented A1 + KEV A2
make test            # 16 pytest cases, all pass
```

4.1 Reproducing on Google Colab

Both my Colab notebook and the live Colab path are documented in the repo. On Colab the CISA live feeds work (sandbox blocked them), so results match my own Oct 29 run of 1,764 KEV entries.

4.2 Verification Artifacts

- Metrics JSON per agent: reports/phase03/agent1_metrics.json, agent2_metrics.json, agent3_metrics.json, agent1_augmented_metrics.json, agent2_kev_metrics.json
- Visualizations: confusion matrices, PR curves, SHAP summary plots as PNGs under reports/phase03/
- Experiment log: reports/phase03/experiments/runs_summary.json consolidates every training run
- Consolidated orchestrator alerts with framework mapping: reports/phase03/consolidated_alerts.json
- Serialized models: models/agent1_rf.joblib, agent1_xgb.joblib, agent1_augmented.joblib, agent2_logistic.joblib, agent2_logistic_kev.joblib, agent3_rf.joblib, agent3_xgb.joblib

5. What Is Next (Phase 04, Weeks 11 and 12)

5.1 Real Data Integration

- Run the full Phase 03 pipeline against HAI 23.05 real data on Colab. The loader is already committed; this is a one-day task
- Stand up OpenPLC in Docker on my home cyber lab machine and point our collectors at the real Modbus TCP endpoints. One-day setup
- Record new baseline metrics on real data and update the Phase 03 report with those numbers

5.2 Operator Interface

- FastAPI orchestrator service with /ingest, /correlate, and /alerts endpoints
- Streamlit operator dashboard showing live alerts with ATT&CK + framework heatmap
- Alert deduplication and severity scoring based on asset criticality

5.3 Deep Sequence Model (Agent 1 Upgrade)

- PyTorch Autoencoder + LSTM for per-packet sequence detection
- Compare against our current XGBoost champion to pick the final production model
- Expect the deep model to help on T0855 Unauthorized Command Message where sequence order matters

5.4 Validation and Delivery

- Red team dry run. Robert attacks, Cesar defends. Swap on pass 2
- Final metrics report with confusion matrices, PR curves, and SHAP summaries on real data
- Scripted live demo. Attack injected on stage, real-time detection, ATT&CK and defensive framework tagging shown on the dashboard
- Final deliverable package. Code, documentation, final deck, 10-minute demo video

5.5 Resources Needed

- One lab machine for OpenPLC Docker (my home cyber lab Linux box is sufficient)
- Google Colab Pro for GPU training of the LSTM (shared Brandon + Cesar)
- No budget asks. All tools used (Python stack, HAI dataset, CISA feeds, ATT&CK STIX, MITRE D3FEND) are free and under permissive or academic-use licenses

6. Risks and Mitigations

- **Risk:** Synthetic data to real data transfer gap. **Mitigation:** we validate against HAI real captures in Phase 04 and hold out two full attack scenarios never seen in training for the final metric.
- **Risk:** Model overfit to simulated attack signatures. **Mitigation:** the realism injections in Track A are already in place. HAI real data in Phase 04 is the final check.
- **Risk:** LSTM compute requirement for Agent 1 upgrade. **Mitigation:** pre-train on the HCC lab VM, then fine-tune on Colab GPU. XGBoost stays as the production fallback if the LSTM does not clear the F1 bar.
- **Risk:** Scope creep across three agents plus orchestrator plus dashboard. **Mitigation:** weekly checkpoint sign-off from both co-leads. The project ships even if the dashboard slips to a notebook demo.

7. Sources

- NIST (2024). Cybersecurity Framework 2.0. https://doi.org/10.6028/NIST.CSWP.29
- NIST (2023). SP 800-82 Rev. 3, Guide to Operational Technology Security. https://doi.org/10.6028/NIST.SP.800-82r3
- IEC (2013). IEC 62443-3-3: System Security Requirements and Security Levels.
- MITRE. ATT&CK for ICS v18 (November 2025). https://attack.mitre.org/matrices/ics/
- MITRE. D3FEND Matrix. https://d3fend.mitre.org/
- CISA. Known Exploited Vulnerabilities Catalog. https://www.cisa.gov/known-exploited-vulnerabilities-catalog
- CISA. ICS Advisories. https://www.cisa.gov/news-events/ics-advisories
- Dragos (2024). 2024 OT Cybersecurity Year in Review. https://www.dragos.com/ot-cybersecurity-year-in-review/
- SANS Institute (2024). ICS/OT Cybersecurity Survey.
- Nozomi Networks (2024). OT/IoT Security Report, H2 2024.
- Shin et al. (2020). HAI 1.0: HIL-based Augmented ICS Security Dataset. https://arxiv.org/abs/2010.05675
- icsdataset/hai GitHub. https://github.com/icsdataset/hai
- Industrial Cyber (2026). Manufacturing absorbs 56% ransomware surge of global attacks in 2025.
- DeNexus (2025). Q3 2025 OT Cyber Incidents.